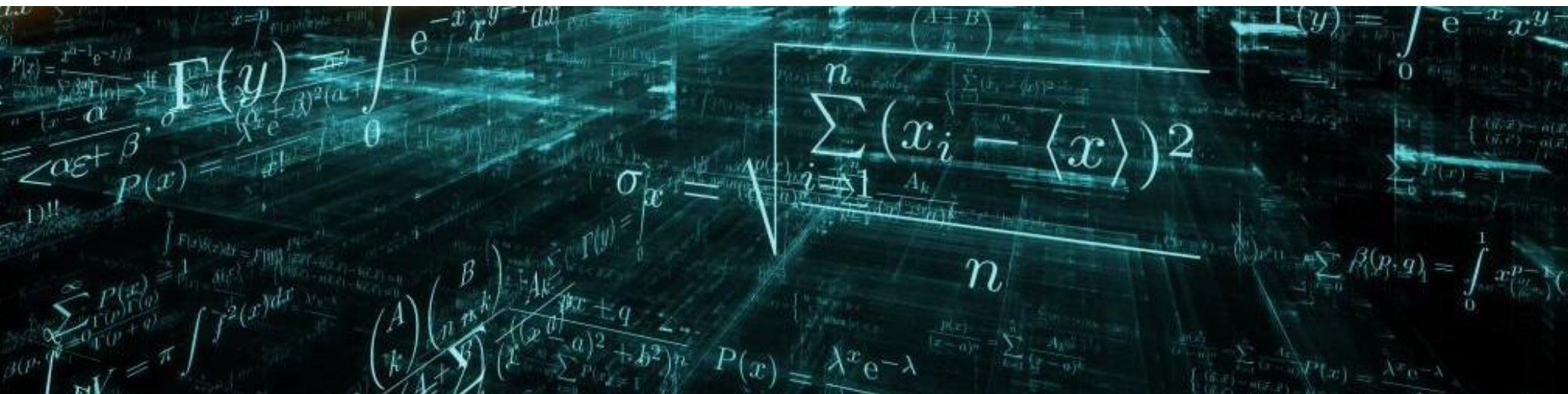


Multimedia Retrieval

Chapter 1: Classical Text Retrieval

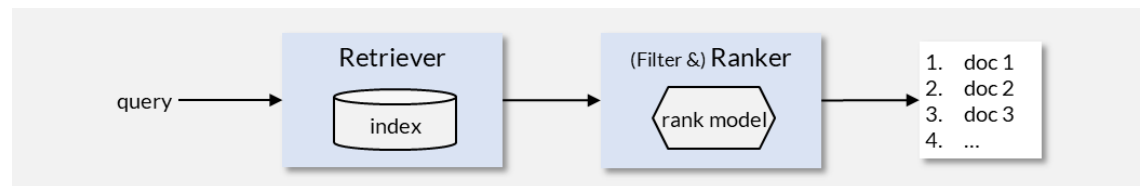
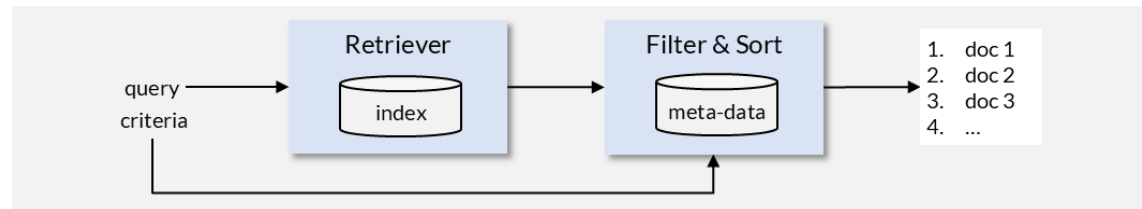
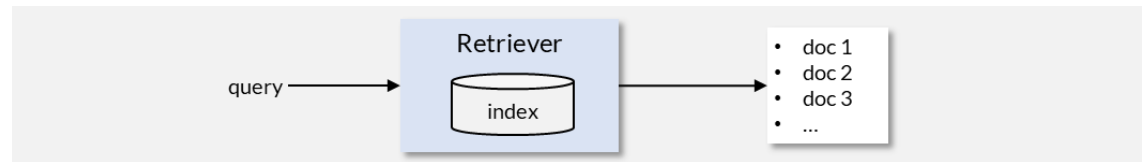
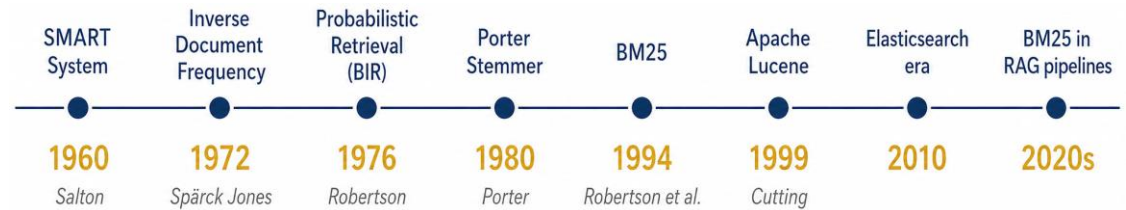
Dr. Roger Weber, roger.weber@gmail.com

Why classical retrieval still matters
From Text to Searchable Vectors
Boolean Retrieval
Vector Space: Query as a Vector
Probabilistic Ranking: BIR



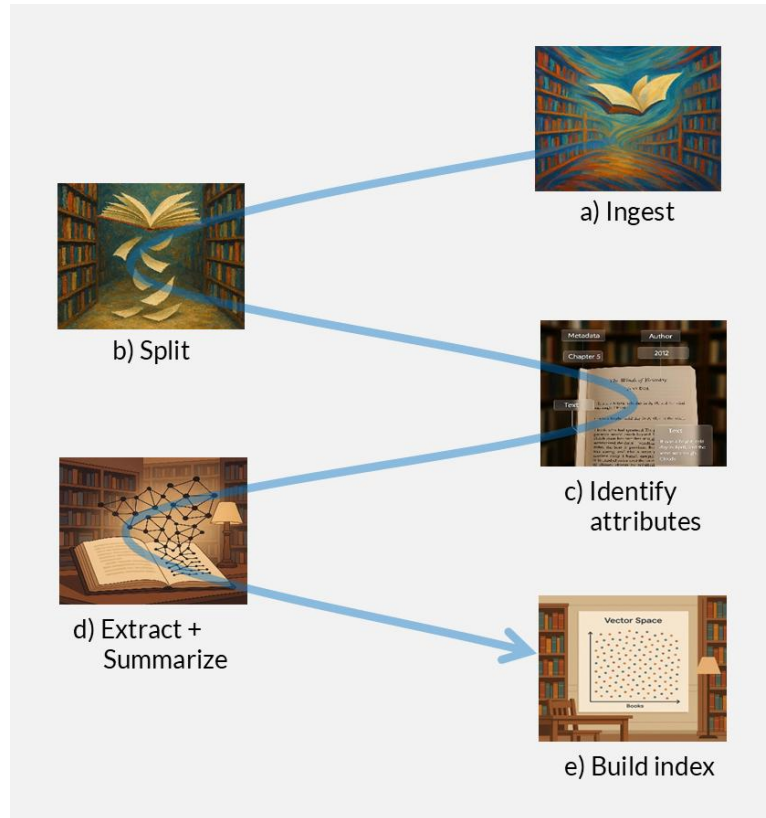
Why classical retrieval still matters

- Given a short query, find the relevant items in a huge collection
 - Text has the smallest semantic gap: query and document share a vocabulary
 - "Solved" by the 2000s - then RAG made retrieval quality matter again
 - BM25, a 30-year-old formula, is the first-stage retriever in modern AI
-
- Three architectures, historically in order
 - Retriever-only: Boolean filter, returns a set
 - faceted filtering: sort/filter by metadata
 - Retriever-ranker: estimate a degree of relevance, order it

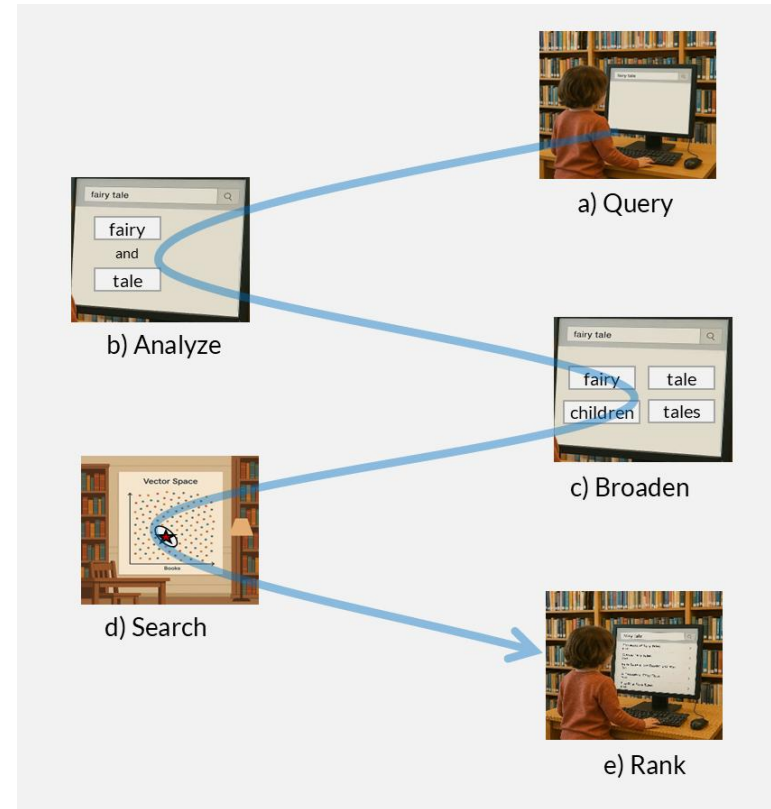


Offline index, online query — same pipeline

- **Offline:** ingest, split, extract features, build index — before any query
- Document = ID + metadata + content
- Retrieval granularity: what is one unit?
 - 800-page book vs "page 47: Ibuprofen dosage",
Finer = specific but more entries

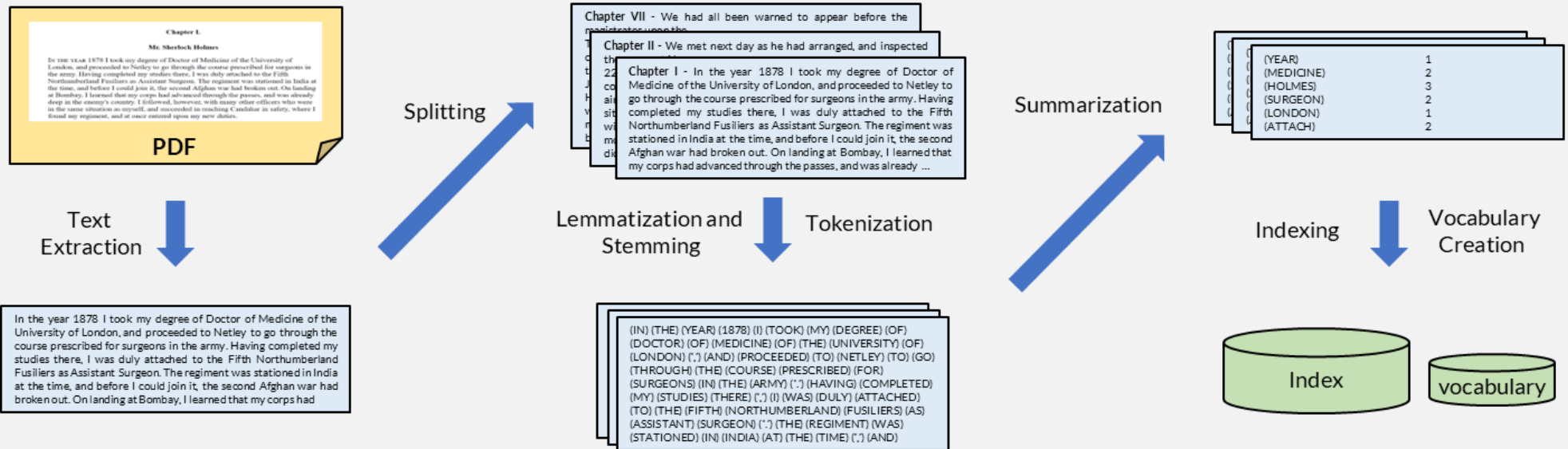


- **Online:** tokenize the query, expand, search, rank
 - Symmetry is essential , Query and documents must live in the same feature space
- Challenge: **relevance ranking**
 - Estimate document importance via feature-query relationship



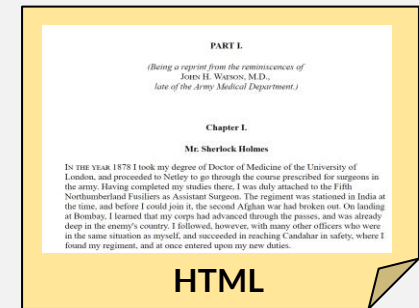
From text to searchable vectors

- Retrieval systems compare concise representations, not PDFs or character streams
- Pipeline: extract, split, tokenize, stem, summarize
- Produces two artifacts;
 - Index (feature vectors) + vocabulary (normalized terms)



Extract Content and Metadata Attributes

- Text formats: HTML, PDF, EPUB, plain text
- Extract meta information and text stream
- Remove control sequences and formatting
- Structural analysis, encoding adjustments, relevant info
- Text extraction from images if needed
- Character encoding: UTF-8/16/32 for indexing



```
<html>
  <head>
    <title> MMIR - 2023 </title>
    <meta name="keywords"
    content="multimedia, retrieval, course"/>
  </head>

  <body>
    ...
    ...
  </body>
</html>
```

Body: Contains the main content enriched with markups. The document's flow is not always obvious and may appear differently on screen than in the file.

Header: Contains meta-information about the document. We can utilize this information to add relevant metadata for the document (and its chunks).

In the year 1878 I took my degree of Doctor of Medicine of the University of London, and proceeded to Netley to go through the course prescribed for surgeons in the army. Having completed my studies there, I was duly attached to the Fifth Northumberland Fusiliers as Assistant Surgeon. The regiment was stationed in India at the time, and before I could join it, the second Afghan war had broken out. On landing at Bombay, I learned that my corps had advanced through the passes, and was already deep in the enemy's country. I ...

Splitting into Chunks

- Traditional retrieval: single term vector per document
 - Short docs: full file acceptable (e.g., library)
 - Large docs: split into smaller chunks that are independent units (e.g., log files)
- Chunking improves precision; increases collection entries
 - Term proximity ignored in traditional models
 - Chunk size trade-off: precision vs performance
 - Adaptive methods: overlapping windows, hierarchical chunking

book-1234.json	book-5678.json	book-9012.json
<pre>title = "Database Systems: The Complete Book" authors = "Garcia-Molina, Ullman & Widom" domain = "Computer Science" year = 2008</pre>	<pre>title = "Operating System Concepts" authors = "Silberschatz, Galvin & Gagne" domain = "Computer Science" year = 2018</pre>	<pre>title = "Introduction to Algorithms" authors = "Cormen, Leiserson, Rivest & Stein" domain = "Computer Science" year = 2009</pre>

orders-app-2026-09-06.log
<pre>> 2026-09-06T08:14:22Z INFO order-service orderId=10432 user=alice event=ORDER_CREATED amount=149.90 currency=CHF items=3</pre>
<pre>> 2026-09-06T08:14:23Z INFO payment-service orderId=10432 user=alice event=PAYMENT_AUTHORIZED method=visa amount=149.90</pre>
<pre>> 2026-09-06T08:14:25Z WARN inventory-svc orderId=10432 sku=BOOK-771 event=LOW_STOCK remaining=2</pre>
<pre>> 2026-09-06T08:14:26Z INFO fulfillment-svc orderId=10432 event=SHIPMENT_SCHEDULED carrier=DHL eta=2026-09-09</pre>
<pre>> 2026-09-06T08:15:01Z ERROR payment-service orderId=10433 user=bob event=PAYMENT_DECLINED reason=insufficient_funds</pre>
<pre>> 2026-09-06T08:15:02Z INFO order-service orderId=10433 user=bob event=ORDER_CANCELLED</pre>

Tokenization

- Tokens: sequences of characters or words
 - **Character/fragment tokens:** handle unseen words, small vocabulary
 - **Words:** classical text retrieval, handle special chars/numbers
 - **Stemming:** merge word variations, control vocabulary
 - **N-grams/phrases:** multi-word tokens, semantic units
- Later: advanced tokenization, embeddings, generative AI

In the year 1878 I took my degree of Doctor of Medicine of the University of London, and proceeded to Netley to go through the course prescribed for surgeons in the army. Having completed my studies there, I was duly attached to the Fifth Northumberland Fusiliers as Assistant Surgeon. The regiment was stationed in India at the time, and before I could join it, the second Afghan war had broken out. On landing at Bombay, I learned that my corps had advanced through the passes, and was already deep in the enemy's country. I ...



(IN) (THE) (YEAR) (1878) (I) (TOOK) (MY)
(DEGREE) (OF) (DOCTOR) (OF) (MEDICINE) (OF)
(THE) (UNIVERSITY) (OF) (LONDON) (',') (AND)
(PROCEEDED) (TO) (NETLEY) (TO) (GO)
(THROUGH) (THE) (COURSE) (PRESCRIBED)
(FOR) (SURGEONS) (IN) (THE) (ARMY) (',')
(HAVING) (COMPLETED) (MY) (STUDIES)
(THERE) (',') (I) (WAS) (DULY) (ATTACHED) (TO)
(THE) (FIFTH) (NORTHUMBERLAND)
(FUSILIERS) (AS) (ASSISTANT) (SURGEON) (',')
(THE) (REGIMENT) (WAS) (STATIONED) (IN)
(INDIA) (AT) (THE) (TIME) (',') (AND) (BEFORE) (I)
(COULD) (JOIN) (IT) (',') (THE) (SECOND)
(AFGHAN) (WAR) (HAD) (BROKEN) (OUT) (',') ...

Lemmatization and Stemming

- Word: informal, human notion, before any processing
- Token: raw unit after splitting, word, number, or punctuation mark
- Term: normalized token after stemming/case-folding → the vocabulary entry
 - Match by identity: "cats" ≠ "cat", "dog" ≠ "hound"
 - Problem: a query for "cats" does not match with a document that contains only "cat"
- Stemming: rule-based suffix stripping, no dictionary
 - Porter developed rule-based approach for stemming
 - Stems are not real words, they only need to collide
- Lemmatization: dictionary based approach to map words to their actual root form (the lemma)

Input	Porter Stem	True Stem (lemma)
“computing”, “computation”, “computer”	comput	compute
“retrieval”, “retrieving”, “retrieved”	retriev	retrieve
“generalization”, “generalizing”	general	generalize
“went”	went	go
“agreed”, “agreeable”	agre	agree

Stop words and Zipf's law

- Top 50 terms ≈ a third of all occurrences, in 60%+ of docs
- Which terms to keep?
 - Frequent terms cannot discriminate
→ so-called stop words
 - The rarest terms rarely appear in queries
 - Best terms sit between the extremes
- Zipf's law: rank terms by frequency
 - High-frequency words: minimal significance, low discriminative power
 - Low-frequency words: rare, may not appear in queries
 - Meaningful words: between upper and lower cut-offs

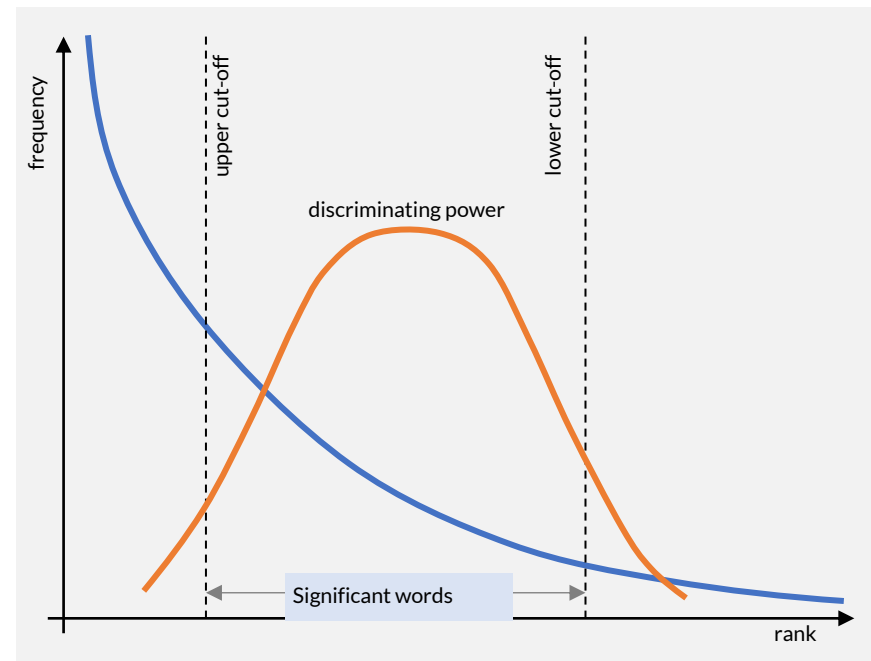
$$p_r = \frac{c}{r} = \frac{tf(t)}{N}, \text{ term } t \text{ with } rank(t) = r$$

c is a constant depending only on M

t_j	$df(t_j)$	$tf(D, t_j)$
the	100%	6.03%
a	99%	2.57%
of	99%	2.55%
and	99%	2.40%
to	99%	2.58%
in	99%	2.00%
for	98%	0.97%
that	96%	1.15%
on	96%	0.75%
is	95%	0.93%
with	95%	0.70%
at	93%	0.55%
by	92%	0.49%
it	92%	0.69%
as	91%	0.57%
but	91%	0.49%

from	90%	0.44%
be	88%	0.46%
an	88%	0.38%
have	88%	0.45%
was	85%	0.65%
not	84%	0.38%
this	83%	0.35%
are	83%	0.45%
has	83%	0.40%
who	81%	0.37%
they	78%	0.37%
he	78%	0.70%
one	77%	0.26%
said	77%	0.70%
more	75%	0.26%
about	75%	0.27%
or	75%	0.31%

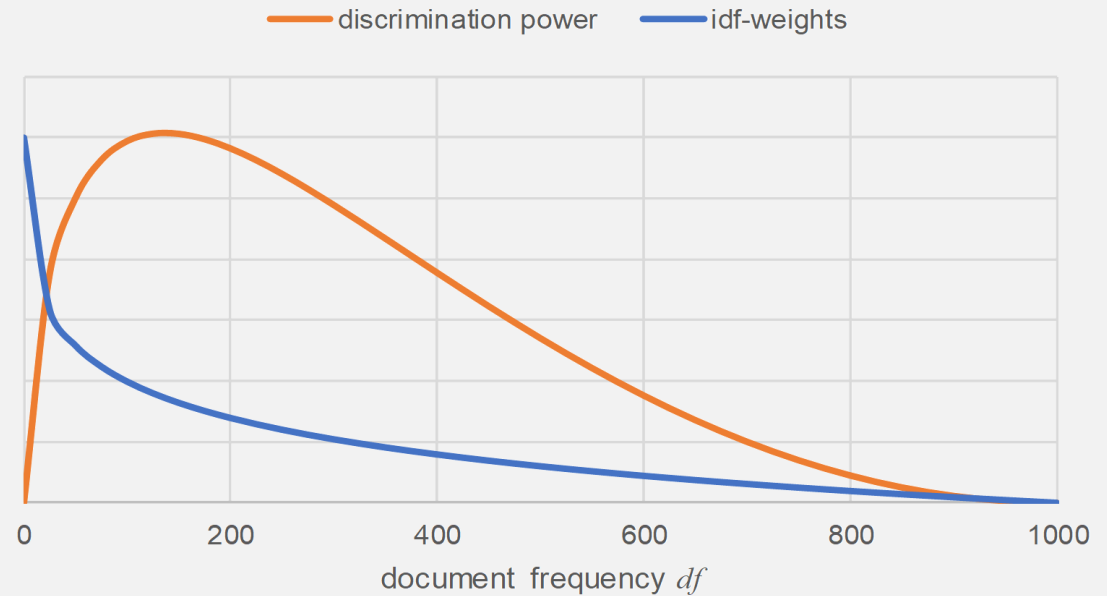
when	74%	0.24%
their	71%	0.27%
his	70%	0.49%
had	70%	0.29%
been	70%	0.21%
all	69%	0.20%
which	69%	0.20%
will	68%	0.27%
out	68%	0.20%
up	68%	0.20%
if	67%	0.21%
than	66%	0.18%
were	66%	0.22%
would	65%	0.23%
can	65%	0.20%
new	64%	0.23%
there	64%	0.18%



IDF: term discrimination in one formula

- Karen Spärck Jones (1972) introduced **inverse document frequency (idf)**
 - $df(t)$: how many documents have term t
 - Rare term $\rightarrow$ high weight
 - Frequent term $\rightarrow$ near zero
- **Discrimination power**
 - $idf(t)^2 \cdot tf(D_i, t) \cdot p(t)$
 - $p(t)$: probability that term occurs in query
 - prefers rare terms that still are likely in queries
 - (almost) eliminates impact of most frequent terms

$$idf(t) = \log \frac{N + 1}{df(t) + 1} = \log(N + 1) - \log(df(t) + 1)$$



Set-of-words, bag-of-words, tf-idf

- Summarization:
 - create concise document feature vectors
 - vocabulary has millions of terms
→ sparse feature vector

- Set-of-words:
 - present (1) or absent (0)

$$d_{i,j} = \begin{cases} 1 & tf(D_i, t_j) > 0 \\ 0 & tf(D_i, t_j) = 0 \end{cases}$$

- Bag-of-words:
 - raw counts

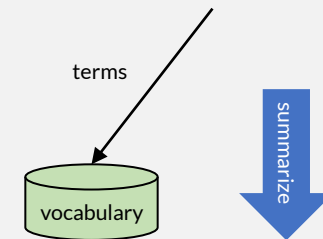
$$d_{i,j} = tf(D_i, t_j)$$

- tf-idf:
 - frequency (tf) × rarity (idf)

$$d_{i,j} = tf(D_i, t_j) \cdot idf(t)$$

- All three ignore order or proximity of terms:
 - "New York" = "York New"

(IN) (THE) (YEAR) (1878) (I) (TOOK) (MY)
(DEGREE) (OF) (DOCTOR) (OF) (MEDICINE) (OF)
(THE) (UNIVERSITY) (OF) (LONDON) (',') (AND)
(PROCEEDED) (TO) (NETLEY) (TO) (GO)
(THROUGH) (THE) (COURSE) (PRESCRIBED)
(FOR) (SURGEONS) (IN) (THE) (ARMY) (',')
(HAVING) (COMPLETED) (MY) (STUDIES)
(THERE) (',') (I) (WAS) (DULY) (ATTACHED) (TO)
(THE) (FIFTH) (NORTHUMBERLAND)
(FUSILIERS) (AS) (ASSISTANT) (SURGEON) (',')
(THE) (REGIMENT) (WAS) (STATIONED) (IN)
(INDIA) (AT) (THE) (TIME) (',') (AND) (BEFORE) (I)
(COULD) (JOIN) (IT) (',') (THE) (SECOND)
(AFGHAN) (WAR) (HAD) (BROKEN) (OUT) (',') ...



(YEAR)	1
(MEDICINE)	2
(HOLMES)	3
(SURGEON)	2
(LONDON)	1
(ATTACH)	2
(UNIVERSITY)	3
(DULY)	1
...	

► [Demo: ch01-01-feature-extraction.ipynb](#)

Boolean retrieval

- A running catalogue of 12 book titles
- Boolean Retrieval
 - Documents are binary vectors (can also write them as set of terms)
 - $D_1 \mapsto [\text{cat}, \text{dog}, \text{forest}, \text{begin}, \text{woodland}, \text{adventure}]$
 - Query = predicate over term existence / absence
 - Each document maps to true / false; no ordering
- Example:
 - cat AND dog $\rightarrow \{D_1, D_9, D_{10}, D_{12}\}$
 - cat OR dog $\rightarrow \{D_1, D_3, D_4, D_7, D_9, D_{10}, D_{12}\}$
 - cat OR (dog AND forest) $\rightarrow \{D_1, D_3, D_4, D_9, D_{10}, D_{12}\}$
 - cat AND NOT dog $\rightarrow \{D_4\}$
- Filters, does not rank $\rightarrow$ all matches equal

ID	Catalogue entry
D_1	The Cat and Dog in the Forest . A cat and a dog begin a woodland adventure.
D_2	Cats of the Woodland . Wild cats begin an adventure beneath ancient trees.
D_3	The Forest Hound. A loyal dog follows a woodland trail through ancient trees.
D_4	Feline Detective . A clever cat solves mysteries with a canine companion.
D_5	Random Forest for Pet Detection. A random forest model classifies cats and dogs in photographs.
D_6	Forests of Search Trees . Algorithms explore a forest of binary trees and graph paths.
D_7	Dog Dog Dog! A dog chases a ball, finds a bone, and wakes the neighbors.
D_8	Forest Forest Forest! A forest guide names trees, flowers, rivers, birds, and hidden ruins.
D_9	Cat Dog Forest .
D_{10}	Cat Dog Forest . A cat meets a dog in a forest beside rivers, mountains, castles, villages, bridges, and caves.
D_{11}	Woodland Companions . A kitten and a puppy share a moonlit adventure among old trees.
D_{12}	The Pet Bakery . A cat , a dog , and a baker make cakes, bread, biscuits, pies, and coffee.

Extended Boolean: soft AND / OR

- Documents represented with bag-of-words ($tf \cdot idf$)

$$d_{i,j} = \min \left(1, \frac{tf(D_i, t_j) \cdot idf(t_j)}{\alpha} \right) \quad \alpha \text{ is fixed}$$

- Still boolean queries, but predicates yield score:

$$sim(\tau_{l,k}, D_i) = \begin{cases} d_{i,j(l,k)} & \text{if } t_{l,k} \text{ must be present} \\ 1 - d_{i,j(l,k)} & \text{if } t_{l,k} \text{ must be absent} \end{cases}$$

- Boolean operator combine values, e.g. p-Norm:

$$sim \left(\bigwedge_{k=1}^K Q_k, D_i \right) = 1 - \sqrt[p]{\frac{\sum_k (1 - sim(Q_k, D_i))^p}{K}}$$

$$sim \left(\bigvee_{k=1}^K Q_k, D_i \right) = \sqrt[p]{\frac{\sum_k sim(Q_k, D_i)^p}{K}}$$

- Example: can AND dog ($p = 2$)

#	D	$d_{i,cat}$	$d_{i,dog}$	AND
1	D_1	0.632	0.500	0.561
2	D_{10}	0.632	0.500	0.561
3	D_7	0.000	1.000	0.293
4	D_9	0.316	0.250	0.282
5	D_{12}	0.316	0.250	0.282
6	D_4	0.316	0.000	0.143
7	D_3	0.000	0.250	0.116

Vector space: query as a vector

- Documents represented with bag-of-words ($tf \cdot idf$)

$$d_{i,j} = tf(D_i, t_j) \cdot idf(t_j)$$

- Collection can be written as term-document matrix **A**

$$\mathbf{A} = \begin{matrix} & \text{document } D_i & \\ \begin{matrix} \left[\begin{array}{ccccc} d_{1,1} & \cdots & d_{i,1} & \cdots & d_{N,1} \\ \vdots & \cdots & \vdots & \cdots & \vdots \\ d_{1,j} & \cdots & d_{i,j} & \cdots & d_{N,j} \\ \vdots & \cdots & \vdots & \cdots & \vdots \\ d_{1,M} & \cdots & d_{i,M} & \cdots & d_{N,M} \end{array} \right] & \text{term } t_j \end{matrix}$$

- Note: Matrix **A** is very sparse with millions of terms (rows) but only few terms per document (columns)

- Queries are also documents!

$$q_j = tf(Q, t_j) \cdot idf(t_j)$$

- Query compared to documents → score

- inner vector product

$$\text{sim}(Q, D_i) = \mathbf{q} \cdot \mathbf{d}_i = \sum_{j=1}^M q_j \cdot d_{i,j}$$

- cosine measure

$$\text{sim}(Q, D_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \cdot \|\mathbf{d}_i\|} = \frac{\sum_{j=1}^M q_j \cdot d_{i,j}}{\sqrt{\sum_{j=1}^M q_j^2} \cdot \sqrt{\sum_{j=1}^M d_{i,j}^2}}$$

Inner product vs cosine measure

- For $Q = \{\text{cat dog forest}\}$ and $N = 12$
 - $idf(cat) = \ln\left(\frac{12}{5}\right) \approx 0.875$
 - $idf(dog) = \ln\left(\frac{12}{6}\right) \approx 0.693$
 - $idf(forest) = \ln\left(\frac{12}{7}\right) \approx 0.539$
- Example evaluation on the right
 - D_9 and D_{10} flip roles when switch measure
 - inner product prefers longer document with frequent query terms
 - cosine measure prefers shorter documents with only query terms (= inner product divided by query length and document length)

#	D	tf (cat, dog, forest)	Inner Product
1	D_{10}	(2, 2, 2)	3.075
2	D_1	(2, 2, 1)	2.784
3	D_7	(0, 4, 0)	1.922
4	D_9	(1, 1, 1)	1.537
5	D_{12}	(1, 1, 0)	1.247
6	D_8	(0, 0, 4)	1.162
7	D_3	(0, 1, 1)	0.771
8	D_4	(1, 0, 0)	0.766
9	D_5	(0, 0, 2)	0.581
10	D_6	(0, 0, 1)	0.291

#	D	tf (cat, dog, forest)	Cosine Measure
1	D_9	(1, 1, 1)	1
2	D_1	(2, 2, 1)	0.659
3	D_{10}	(2, 2, 2)	0.342
4	D_7	(0, 4, 0)	0.232
5	D_8	(0, 0, 4)	0.139
6	D_{12}	(1, 1, 0)	0.137
7	D_3	(0, 1, 1)	0.102
8	D_4	(1, 0, 0)	0.093
9	D_5	(0, 0, 2)	0.059
10	D_6	(0, 0, 1)	0.034

Probabilistic Ranking: BIR

- So far, models have used heuristics
- Probabilistic retrieval asks a different question:
 - given query Q and document D_i , **how likely is the document to be relevant?**

$$\text{sim}(Q, D_i) = \frac{P(R|D_i)}{P(NR|D_i)} = \frac{P(R|D_i)}{1 - P(R|D_i)}$$

- Binary Independence Model (BIR) → assumptions to compute the above probabilities
 1. Term frequency does not matter (set-of-words)
 2. Term independence (like all models before)
 3. Only query terms influence ranking
(if a term is absent from the query, it's assumed to be equally distributed among relevant and non-relevant documents)

- Apply Bayes theorem

$$\text{sim}(Q, D_i) = \frac{P(R|D_i)}{P(NR|D_i)} = \frac{P(D_i|R) \cdot P(R)}{P(D_i|NR) \cdot P(NR)}$$

- Using assumptions 1&2, $P(D_i|R)$ is (similar $P(D_i|NR)$):

Assumption 1:
Documents are
binary vectors

Assumption 2: Terms
are independent

$$P(D_i|R) = P(d_i|R) = \prod_{j=1}^M P(d_{i,j}|R)$$

$$P(D_i|R) = \prod_{\forall j: d_{i,j}=1} P(d_{i,j} = 1|R) \cdot \prod_{\forall j: d_{i,j}=0} P(d_{i,j} = 0|R)$$

Assumption 1: Documents
are binary vectors

- Let $r_j = P(d_{i,j} = 1|R)$ and $n_j = P(d_{i,j} = 1|NR)$

$$\text{sim}(Q, D_i) = \frac{P(R)}{P(NR)} \cdot \prod_{\forall j: d_{i,j}=1} \frac{r_j}{n_j} \cdot \prod_{\forall j: d_{i,j}=0} \frac{1 - r_j}{1 - n_j}$$

$$\text{sim}(Q, D_i) \sim \prod_{\forall j: d_{i,j}=1} \frac{r_j}{n_j} \cdot \prod_{\forall j: d_{i,j}=0} \frac{1 - r_j}{1 - n_j}$$

Calculating Similarity Scores with BIR

- Assumption 3 reduces products to query term

$$\text{sim}(Q, D_i) \sim \prod_{\forall j: d_{i,j}=1, q_j=1} \frac{r_j}{n_j} \cdot \prod_{\forall j: d_{i,j}=0, q_j=1} \frac{1-r_j}{1-n_j}$$

- If we drop conditions $d_{i,j} = 0$ in the right product, we need to compensate for $d_{i,j} = 1$ in the left product

$$\text{sim}(Q, D_i) \sim \prod_{\forall j: d_{i,j}=1, q_j=1} \frac{r_j \cdot (1-n_j)}{n_j \cdot (1-r_j)} \cdot \prod_{\forall j: q_j=1} \frac{1-r_j}{1-n_j}$$

- The right product only depends on query

$$\text{sim}(Q, D_i) \sim \prod_{\forall j: d_{i,j}=1, q_j=1} \frac{r_j \cdot (1-n_j)}{n_j \cdot (1-r_j)}$$

- Using logarithm, simplifies to the final form:

$$\text{sim}(Q, D_i) \sim \sum_{\forall j: d_{i,j}=1, q_j=1} c_j \quad \text{with } c_j = \log \frac{r_j \cdot (1-n_j)}{n_j \cdot (1-r_j)}$$

- But how do we calculate the c_j -values?

- $r_j = P(d_{i,j} = 1 | R) \rightarrow$ probability that a relevant document contains the term t_j
- $n_j = P(d_{i,j} = 1 | NR) \rightarrow$ probability that a non-relevant document contains the term t_j
- If we would know a sample of relevant/non-relevant documents, we could simply count

- Initial estimates:

$$r_j = 0.5, \quad n_j = \frac{df(t_j) + 0.5}{N + 1} \quad \forall j: q_j = 1$$

- Produces a first result list. Let user mark L documents as relevant (out of K retrieved documents)
- If term t_j appears in l_j relevant and k_j retrieved documents, we can estimate c_j based on this sample:

$$r_j = \frac{l_j + 0.5}{L + 1}, \quad n_j = \frac{k_j - l_j + 0.5}{K - L + 1} \quad \forall j: q_j = 1$$

BIR Example with Feedback

- Query: **dog forest**
- All 12 documents retrieved, D_1 and D_3 marked relevant

Term	l_j	k_j	r_j	n_j	c_j
dog	2	6	0.833	0.409	1.977
forest	2	7	0.833	0.500	1.609

- In addition, we can expand query with terms that separate relevant from non-relevant documents

Term	l_j	k_j	r_j	n_j	c_j
woodland	2	4	0.833	0.227	2.833
algorithms	0	1	0.167	0.136	0.236
cats	0	2	0.167	0.227	-0.386

- "woodland" → good candidate for query expansion
- "algorithms" → not able to separate enough
- "cats" → candidate to eliminate false positives

ID	Catalogue entry
D_1	The Cat and Dog in the Forest . A cat and a dog begin a woodland adventure.
D_2	Cats of the Woodland . Wild cats begin an adventure beneath ancient trees.
D_3	The Forest Hound. A loyal dog follows a woodland trail through ancient trees.
D_4	Feline Detective . A clever cat solves mysteries with a canine companion.
D_5	Random Forest for Pet Detection. A random forest model classifies cats and dogs in photographs.
D_6	Forests of Search Trees . Algorithms explore a forest of binary trees and graph paths.
D_7	Dog Dog Dog! A dog chases a ball, finds a bone, and wakes the neighbors.
D_8	Forest Forest Forest! A forest guide names trees, flowers, rivers, birds, and hidden ruins.
D_9	Cat Dog Forest .
D_{10}	Cat Dog Forest . A cat meets a dog in a forest beside rivers, mountains, castles, villages, bridges, and caves.
D_{11}	Woodland Companions . A kitten and a puppy share a moonlit adventure among old trees.
D_{12}	The Pet Bakery . A cat, a dog , and a baker make cakes, bread, biscuits, pies, and coffee.

BM25: Fixing Repetition without limit

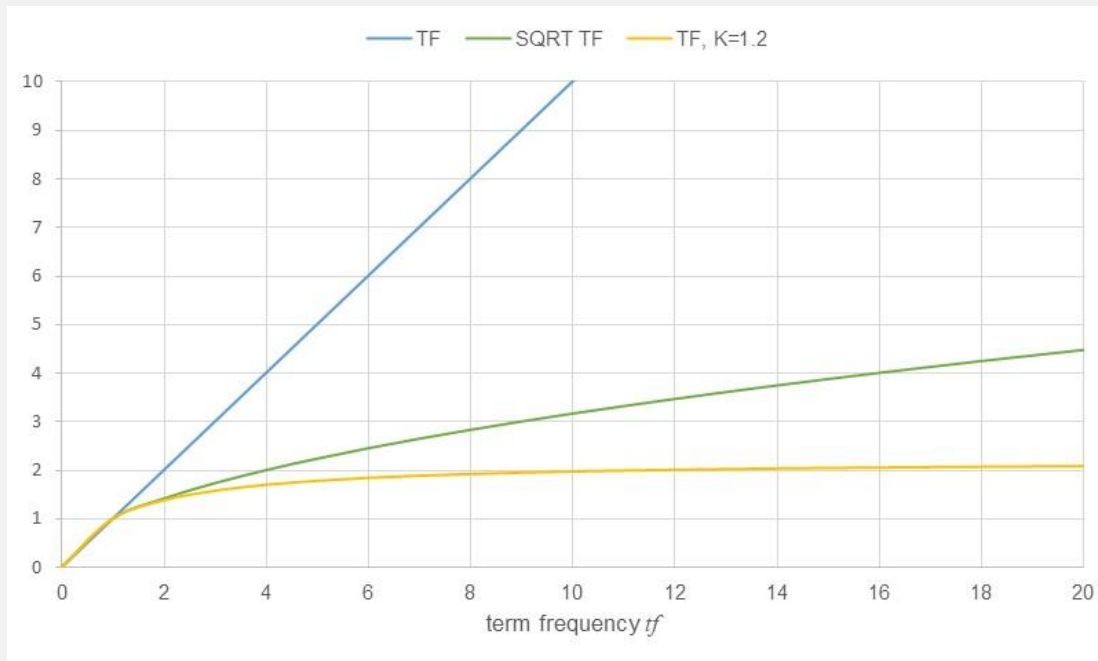
- Inner product score grows linearly
 - invites keyword stuffing (spamming)
 - how much better are 1000 occurrences of the query term compared to 100 occurrences
 - inner product → 10x better
 - cosine measure → normalization by length
 - strong preference for longer documents

- **Idea:** fast saturation of term frequency
 - Not just using log or sqrt to slow down, but introduce upper ceiling with parameter k as steering

$$\widehat{tf}_k = \frac{tf \cdot (k + 1)}{tf + k}$$

($k+1$) scales values but does not impact ranking

- Typically, $k \in [1, 2]$ with Lucene using $k = 1.2$
- Ceiling at $\max \widehat{tf}_k = k + 1$



BM25: Fixing Preference for Long Documents

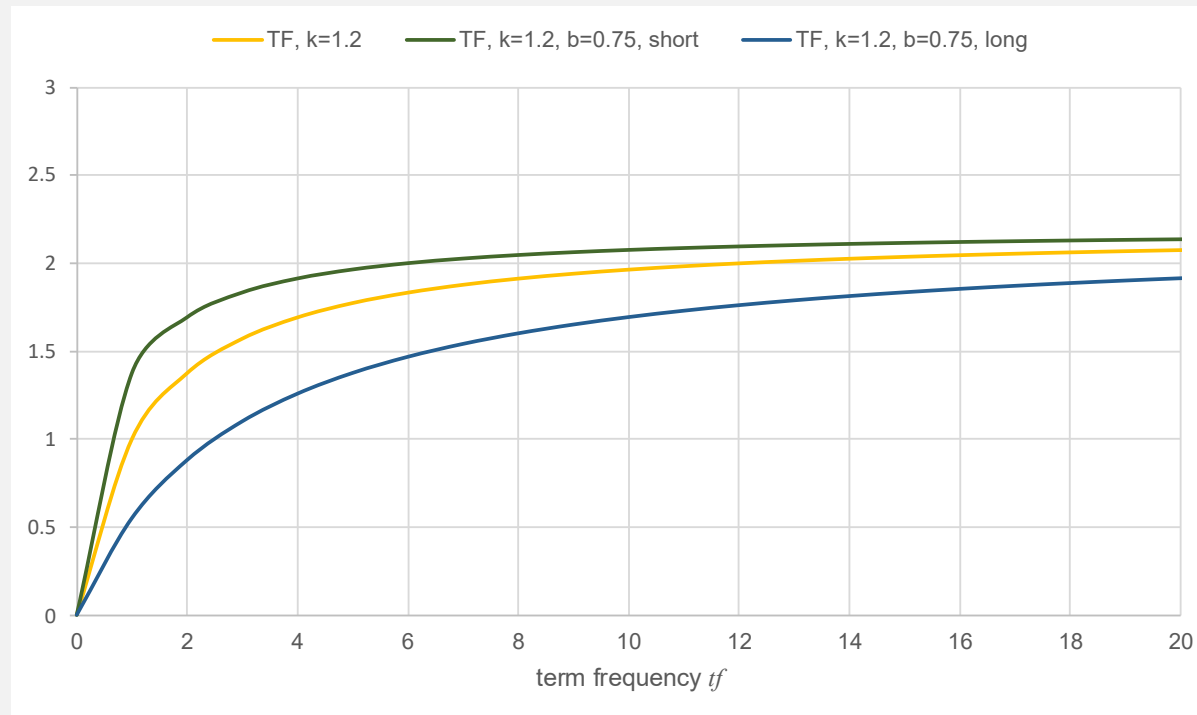
- Inner product prefers longer documents
 - long documents likely contain query terms more often than short documents
 - cosine measure penalizes longer documents with the length normalization

- Idea: Normalize term frequency further
 - Compare document length $|D|$ with average document length $avgdl$ to boost short documents

$$\widehat{tf}_k(D) = \frac{tf \cdot (k + 1)}{tf + k \cdot \left(1 - b + b \frac{|D|}{avgdl}\right)}$$

avgdl is the average document length

- with $b = 0.75$ (adjustable)



BM25: Using BIR's Idea for Term Weighting

- BIR's initial guess was based on:

$$r_j = 0.5, \quad n_j = \frac{df(t_j) + 0.5}{N + 1} \quad \forall j: q_j = 1$$

- leads to:

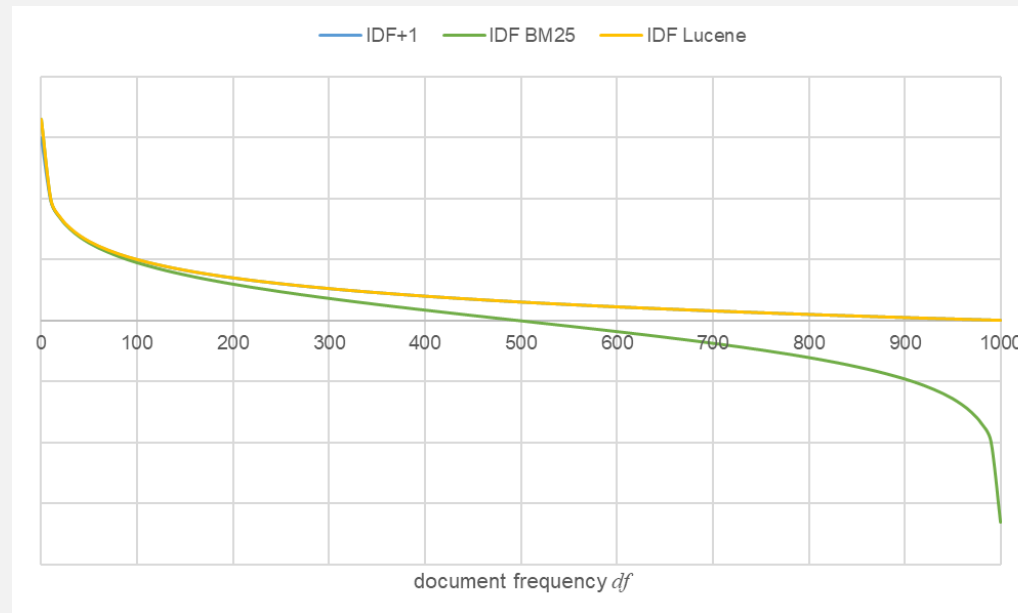
$$c_j = \log \frac{N - df(t_j) + 0.5}{df(t_j) + 0.5} = idf_{BM25}(t_j)$$

- With that, BM25 scores are as follows

$$sim_{BM25}(Q, D_i) = \sum_{j=1}^M \log \left(\frac{N - df(t_j) + 0.5}{df(t_j) + 0.5} \right) \cdot \widehat{tf}_k(D)$$

- Lucene implements BM25 with different IDF function to avoid negative values

$$idf_{lucene}(t_j) = \log \left(\frac{N+1}{df(t_j)+0.5} \right)$$



BIR Example

- For $Q = \{\text{cat dog forest}\}$ and $N = 12$
 - Right side shows results with vector space model
 - Table below shows BM25 scoring
 - D_9 (short) ranks highest thanks to term frequency normalization, but D_{10} (long) is close
 - in contrast: both inner product and cosine measure were not able to score them close to each other

#	D	Length	tf (cat, dog, forest)	BM25 score
1	D_9	3	(1, 1, 1)	2.930
2	D_1	8	(2, 2, 1)	2.836
3	D_{10}	14	(2, 2, 2)	2.568
4	D_{12}	11	(1, 1, 0)	1.470
5	D_3	10	(0, 1, 1)	1.226
6	D_7	10	(0, 4, 0)	1.166
7	D_4	8	(1, 0, 0)	0.925
8	D_8	12	(0, 0, 4)	0.894
9	D_5	11	(0, 0, 2)	0.728
10	D_6	10	(0, 0, 1)	0.542

#	D	tf (cat, dog, forest)	Inner Product
1	D_{10}	(2, 2, 2)	3.075
2	D_1	(2, 2, 1)	2.784
3	D_7	(0, 4, 0)	1.922
4	D_9	(1, 1, 1)	1.537
5	D_{12}	(1, 1, 0)	1.247
6	D_8	(0, 0, 4)	1.162
7	D_3	(0, 1, 1)	0.771
8	D_4	(1, 0, 0)	0.766
9	D_5	(0, 0, 2)	0.581
10	D_6	(0, 0, 1)	0.291

#	D	tf (cat, dog, forest)	Cosine Measure
1	D_9	(1, 1, 1)	1
2	D_1	(2, 2, 1)	0.659
3	D_{10}	(2, 2, 2)	0.342
4	D_7	(0, 4, 0)	0.232
5	D_8	(0, 0, 4)	0.139
6	D_{12}	(1, 1, 0)	0.137
7	D_3	(0, 1, 1)	0.102
8	D_4	(1, 0, 0)	0.093
9	D_5	(0, 0, 2)	0.059
10	D_6	(0, 0, 1)	0.034

► Demo: [ch01-02-retrieval-models.ipynb](#)

Summary

Property	Boolean	Extended Boolean	Vector Space	BIR	BM25
Query form	Boolean expression	Boolean expression	Free text	Term set	Free text
Ranked output	No	Yes	Yes	Yes	Yes
Partial matches	No	Yes	Yes	Yes	Yes
Term frequency	Ignored	Normalized TF-IDF	Linear TF-IDF	Ignored	Saturating
Document length	Ignored	Indirect normalization	Cosine or none	Ignored	Explicit parameter b
Foundation	Set theory	Soft-logic heuristic	Geometric heuristic	Relevance probability	BIR plus TF and length models
Typical role	Exact filters	Specialized soft constraints	Baseline and vector similarity	Feedback model and foundation	Production lexical ranking
Main weakness	No ranking	Operator choices	Length and repetition effects	Binary evidence and feedback burden	Lexical matching and tuning

Further Reading

- Spärck Jones, K. (1972). **A Statistical Interpretation of Term Specificity and Its Application in Retrieval.** *Journal of Documentation*, 28(1), 11-21. [PDF](#). Introduced inverse document frequency; foundational for every term-weighting scheme discussed in this chapter, including TF-IDF and BM25.
- Salton, G., Wong, A., & Yang, C. S. (1975). **A Vector Space Model for Automatic Indexing.** *Communications of the ACM*, 18(11), 613-620. [PDF](#). The original paper introducing the Vector Space Model; still readable and surprisingly concise.
- Robertson, S. E., & Spärck Jones, K. (1976). **Relevance Weighting of Search Terms.** *Journal of the American Society for Information Science*, 27(3), 129-146. [Publisher page](#). Formalizes the Binary Independence Model and its r_j/n_j relevance-feedback estimates covered in this chapter.
- Salton, G., Fox, E. A., & Wu, H. (1983). **Extended Boolean Information Retrieval.** *Communications of the ACM*, 26(11), 1022-1036. [Free access](#). Introduces the p-norm model, positioning graded Boolean retrieval between the strict Boolean and vector-space models.
- Porter, M. F. (1980). **An Algorithm for Suffix Stripping.** *Program*, 14(3), 130-137. [Author's page](#). The classic stemming algorithm still used as a baseline in English text processing.
- Robertson, S. E., & Walker, S. (1994). **Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval.** In *Proceedings of the 17th ACM SIGIR Conference*, 232-241. [PDF](#). Derives BM25's term-frequency saturation and length normalization from the 2-Poisson model discussed in the Probabilistic Ranking and BM25 section.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to Information Retrieval.** Cambridge University Press. [Free online edition](#). The standard graduate textbook; chapters 1 to 6 cover Boolean, VSM, and probabilistic models with rigorous treatment.
- Robertson, S. E., & Zaragoza, H. (2009). **The Probabilistic Relevance Framework: BM25 and Beyond.** *Foundations and Trends in Information Retrieval*, 3(4), 333-389. [PDF](#). Definitive survey of BM25's theoretical foundations and practical extensions.